ShopEZ

Women's Fashion E-Commerce Platform

BACKEND DOCUMENTATION

Microservices Architecture | .NET 8 | ASP.NET Core | SQL Server

Version 1.0 | May 2026

1. Problem Statement

The modern Indian woman demands a sophisticated, trustworthy digital shopping experience. Existing platforms either lack curated women's fashion categories or fail to deliver the seamless, secure, and fast experience women expect. ShopEZ was built to address this gap — a dedicated women's fashion e-commerce platform inspired by the success of Nykaa Fashion, tailored for Indian consumers.

1.1 Core Pain Points Addressed

- Lack of a dedicated women's fashion marketplace with curated categories (Sarees, Anarkali, Jewellery, Footwear, Handbags, Skincare, Summer, Accessories)
- No intelligent inventory labelling — products with low stock should show 'Hot' badges to create urgency; high-stock products should be labelled 'New' to indicate availability
- Insecure payment flows and poor form validation leading to cart abandonment
- No free delivery offer driving purchase decisions for women shoppers
- Fragile monolithic backends unable to scale individual features independently
- Missing wishlist functionality, causing users to lose track of desired items

1.2 Solution: ShopEZ Platform

- Microservices backend with independent AuthService, ProductService, and OrderService
- JWT + Refresh Token authentication for secure, persistent sessions
- Cloudinary-powered image management for fast product image delivery
- Multi-step checkout with address validation, UPI, card, net banking, and COD support
- Stock-based dynamic labels: Low stock (1-5 units) = 'HOT', High stock (6+) = 'NEW'
- Free delivery promotion, wishlist management, category filters, and live search autocomplete

2. System Architecture

ShopEZ follows a microservices architecture pattern where each service is independently deployable, scalable, and maintainable. All client requests are routed through a centralized API Gateway powered by Ocelot.

2.1 High-Level Architecture

Layer	Technology	Responsibility
Frontend	Angular 21	SPA consuming API Gateway
API Gateway	Ocelot + ASP.NET Core	Route requests, CORS, JWT forwarding
Auth Service	ASP.NET Core + SQL Server	User registration, login, JWT/Refresh tokens
Product Service	ASP.NET Core + SQL Server	Product CRUD, Cloudinary image upload
Order Service	ASP.NET Core + SQL Server	Order creation, management, product validation
Database	MS SQL Server 2022	Separate DB per service (AuthDB, ProductDB, OrderDB)
Image Storage	Cloudinary CDN	Product image storage and delivery
Container	Docker + Docker Compose	Orchestration of all services

2.2 Service Port Mapping

Service	Internal Port	External Port	Description
API Gateway	8080	5000	Main entry point for all clients
Auth Service	8080	5003	Authentication & authorization
Product Service	8080	5001	Product catalogue management
Order Service	8080	5002	Order processing & history
SQL Server	1433	1433	Shared DB host, isolated databases
Frontend	4200	4200	Angular development server

2.3 Request Flow

1. Angular frontend sends HTTP request with JWT Bearer token in header
2. API Gateway (port 5000) receives request and matches route via ocelot.json
3. Gateway forwards request to the appropriate downstream microservice

4. Microservice validates JWT, processes business logic, queries its SQL database
5. Response flows back through Gateway to the frontend

3. Microservices Documentation

3.1 API Gateway

The API Gateway is the single entry point for all client requests. Built with Ocelot middleware on ASP.NET Core, it handles request routing, CORS policy, and Authorization header forwarding to downstream services.

3.1.1 Technology Stack

- ASP.NET Core Web Application
- Ocelot API Gateway Middleware
- JWT Bearer Authentication (Microsoft.AspNetCore.Authentication.JwtBearer)
- CORS policy: AllowAll (credential-aware, open origin)

3.1.2 Route Configuration (ocelot.json)

Upstream Path	Method	Downstream Service	Port	Priority
/api/auth/{everything}	GET, POST, PUT, DELETE	authservice	8080	1
/api/products/upload	POST	productservice	8080	5
/api/products/upload/{id}	PUT	productservice	8080	5
/api/products	GET	productservice	8080	4
/api/products	POST	productservice	8080	3
/api/products/{id}	GET	productservice	8080	3
/api/products/{id}	PUT, DELETE	productservice	8080	2
/api/orders	GET, POST, PUT, DELETE	orderservice	8080	2
/api/orders/{everything}	GET, POST, PUT, DELETE	orderservice	8080	1

Note: Order routes include DownstreamHeaderTransform to forward the Authorization header to the Order Service for JWT validation.

3.2 Auth Service

Handles all authentication and user management. Uses BCrypt for password hashing, JWT for access tokens (1-day expiry), and cryptographically random refresh tokens (7-day expiry) stored in SQL Server.

3.2.1 Technology Stack

- ASP.NET Core Web API (.NET 8)
- Entity Framework Core with SQL Server (AuthDB)
- BCrypt.Net for password hashing
- Microsoft.IdentityModel.Tokens for JWT generation
- Swagger/OpenAPI with Bearer auth
- FluentValidation for request validation

3.2.2 Data Model: User

Field	Type	Constraints	Description
UserId	int	Primary Key, Auto-increment	Unique user identifier
Name	string	Required	Full display name
Email	string	Required, EmailAddress	Login credential and unique identifier
Password	string	Required, BCrypt hashed	Never stored in plain text
Role	string	Required, Default: 'User'	'User' or 'Admin'
RefreshToken	string?	Nullable	Cryptographically secure random token
RefreshTokenExpiry	DateTime?	Nullable	7 days from last login/refresh

3.2.3 API Endpoints

Method	Endpoint	Auth	Description	Response
POST	/api/auth/register	None	Register new user (Name, Email, Password, Role)	200 OK or 409 Conflict
POST	/api/auth/login	None	Login with email & password	200 with Token + RefreshToken + User
POST	/api/auth/refresh-token	None	Exchange refresh token for new JWT	200 with new Token + RefreshToken
POST	/api/auth/logout	None	Invalidate refresh token	200 OK
GET	/api/auth/me	JWT Bearer	Get current logged-in user profile	200 with UserDTO

3.2.4 JWT Configuration

- Algorithm: HMAC-SHA256
- Claims: nameid (UserId), unique_name (Name), email, role
- Access Token Expiry: 24 hours
- Refresh Token Expiry: 7 days
- Issuer: MyApp | Audience: MyAppUsers

3.2.5 Security Features

- BCrypt password hashing with automatic salt generation
- MapInboundClaims = false to prevent claim mapping conflicts
- Refresh token rotation on every refresh (old token invalidated)
- Logout clears refresh token from database
- Auto database migration on startup

3.3 Product Service

Manages the product catalogue including CRUD operations and Cloudinary-based image upload. Implements Repository pattern for clean data separation. Admin-only for write operations; public GET endpoints for all users.

3.3.1 Technology Stack

- ASP.NET Core Web API (.NET 8)
- Entity Framework Core with SQL Server (ProductDB)
- Cloudinary .NET SDK for image hosting
- Repository Pattern (IProductRepository / ProductRepository)
- JWT Bearer Authentication with Role-based Authorization

3.3.2 Data Model: Product

Field	Type	Validation	Description
ProductId	int	PK, Auto-increment	Unique product identifier
Name	string	Required	Product display name
Description	string	Required	Detailed product description
Price	decimal	Range: 1 to 1,000,000	Product price in INR
ImageUrl	string	Optional	Cloudinary CDN URL or external URL
Stock	int	Range: 0 to MaxInt	Current inventory level
Category	string	Optional	Women's fashion category

3.3.3 Stock-Based Labelling Logic

The frontend uses Stock values to determine product badges:

- Stock = 0: 'Out of Stock' — add to cart is disabled
- Stock 1-5 (Low): Labelled 'HOT' with red badge — creates urgency for shoppers
- Stock 6+ (High): Labelled 'NEW' with green badge — signals fresh inventory

3.3.4 API Endpoints

Method	Endpoint	Auth	Content-Type	Description
GET	/api/products	None	—	List all products
GET	/api/products/{id}	None	—	Get product by ID
POST	/api/products	Admin JWT	application/json	Create product with image URL
POST	/api/products/upload	Admin JWT	multipart/form-data	Create product with file upload to Cloudinary
PUT	/api/products/{id}	Admin JWT	application/json	Update product (JSON)
PUT	/api/products/upload/{id}	Admin JWT	multipart/form-data	Update product with new image upload
DELETE	/api/products/{id}	Admin JWT	—	Delete product (Admin only)

3.3.5 Cloudinary Integration

- Images auto-transformed to 400x400px (fill crop, gravity: auto)
- Stored in 'ecommerce-shopez' Cloudinary folder
- Secure HTTPS URLs returned and stored in ProductDB
- Supports JPEG, PNG, WebP formats via IFormFile

3.3.6 Repository Pattern

- IProductRepository: interface defining GetAllAsync, GetByIdAsync, AddAsync, UpdateAsync, DeleteAsync
- ProductRepository: EF Core implementation using ProductDbContext
- ProductServiceImpl: business logic layer sitting between controller and repository
- Validation in service layer: price > 0, stock >= 0, name required

3.4 Order Service

Handles order creation, retrieval, and management. Calls Product Service internally to validate products and fetch current pricing. All endpoints require JWT authentication. Users can only view their own orders; Admins can view and manage all orders.

3.4.1 Technology Stack

- ASP.NET Core Web API (.NET 8)
- Entity Framework Core with SQL Server (OrderDB)
- HttpClient (ProductServiceClient) for inter-service communication
- JWT Bearer Authentication with Role-based Access Control

3.4.2 Data Models

Order:

Field	Type	Description
OrderId	int	Primary key, auto-increment
UserId	int	Extracted from JWT claims (nameid)
OrderDate	DateTime	UTC timestamp at order creation
TotalAmount	decimal	Calculated sum of all line item totals
OrderItems	ICollection<OrderItem>	Navigation property to line items

OrderItem:

Field	Type	Description
OrderItemId	int	Primary key
OrderId	int (FK)	Foreign key to Order
ProductId	int	Product reference from ProductService
ProductName	string	Snapshot from ProductService at order time
ProductImageUrl	string	Snapshot of product image at order time
Quantity	int	Range: 1-100
Price	decimal	Price per unit from ProductService at order time

3.4.3 API Endpoints

Method	Endpoint	Auth	Description
POST	/api/orders	User/Admin JWT	Create new order (items from cart)
GET	/api/orders	User/Admin JWT	User: own orders only | Admin: all orders
GET	/api/orders/{id}	User/Admin JWT	Get order by ID (User: own only)
PUT	/api/orders/{id}	Admin JWT	Update order (Admin only)
DELETE	/api/orders/{id}	Admin JWT	Delete order (Admin only)

3.4.4 Inter-Service Communication

The ProductServiceClient calls ProductService's GET /api/products/{id} endpoint during order creation to:

- Validate the product exists
- Fetch the current authoritative price (prevents price manipulation from client)
- Capture product name and image URL as an order snapshot
- Forward the user's Bearer token for authenticated requests

4. Database Design

4.1 Database-per-Service Pattern

Each microservice owns its database schema. No cross-service joins or shared tables. This ensures loose coupling and independent scalability.

Database	Service	Key Tables	Auto-migrated
AuthDB	AuthService	Users (UserId, Name, Email, Password, Role, RefreshToken, RefreshTokenExpiry)	Yes
ProductDB	ProductService	Products (ProductId, Name, Description, Price, ImageUrl, Stock, Category)	Yes
OrderDB	OrderService	Orders, OrderItems (cascade delete)	Yes

4.2 Connection Strings (Docker)

All services connect to the shared SQL Server container using named host 'sqlserver':

- AuthDB: Server=sqlserver;Database=AuthDB;User Id=sa;Password=YourStrong@Pass123;TrustServerCertificate=True
- ProductDB: Server=sqlserver;Database=ProductDB;...
- OrderDB: Server=sqlserver;Database=OrderDB;...

4.3 Auto-Migration

All three services run db.Database.Migrate() on startup. This automatically applies any pending EF Core migrations, creating tables if they do not exist. No manual SQL scripts required.

5. Docker & Deployment

5.1 Docker Architecture

The application is fully containerized using Docker Compose. Each microservice has its own Dockerfile; the docker-compose.yml at the project root orchestrates all six services.

5.2 Service Dependency Graph

- sqlserver starts first (no dependencies)
- authservice, productservice, orderservice: all depend on sqlserver
- apigateway depends on authservice, productservice, orderservice
- frontend depends on apigateway

5.3 Docker Compose Configuration

Service	Image/Build	Container Name	Host Port	Container Port	Env Variable
sqlserver	mcr.microsoft.com/mssql/server:2022-latest	sqlserver	1433	1433	SA_PASSWORD=Your ACCEPT_EULA=Y
authservice	./EcommerceApp/AuthService	authservice	5003	8080	ASPNETCORE_URLS=
productservice	./EcommerceApp/ProductService	productservice	5001	8080	ASPNETCORE_URLS=
orderservice	./EcommerceApp/OrderService	orderservice	5002	8080	ASPNETCORE_URLS=
apigateway	./EcommerceApp/ApiGateway	apigateway	5000	8080	ASPNETCORE_URLS=
frontend	./frontend/ecommerce-angular	frontend	4200	4200	—

5.4 Step-by-Step Docker Deployment

6. Prerequisites: Install Docker Desktop (Windows/Mac) or Docker Engine + Compose (Linux)
7. Clone the repository to your local machine
8. Navigate to: cd Final_working_capstone_project/Angular_Project_4
9. Set Cloudinary credentials in ProductService/appsettings.json (CloudName, ApiKey, ApiSecret)
10. Run: docker compose up --build
11. Wait for SQL Server to initialize (~30s) before services connect
12. Access frontend at: http://localhost:4200
13. Access API Gateway at: http://localhost:5000
14. Access individual services via Swagger: http://localhost:5001, :5002, :5003

5.5 Stopping the Application

- Stop all containers: docker compose down
- Stop and remove volumes (fresh DB): docker compose down -v
- View logs: docker compose logs -f [service-name]
- Rebuild single service: docker compose up --build [service-name]

5.6 Environment Variables Reference

Variable	Service	Value	Description
SA_PASSWORD	sqlserver	YourStrong@Pass123	SQL Server SA password (change in production)
ACCEPT_EULA	sqlserver	Y	Accept SQL Server EULA
ASPNETCORE_URLS	All .NET services	http://+:8080	Bind to all interfaces on port 8080
Jwt:Key	AuthService	ThisIsAVery...12345	HMAC-SHA256 signing secret — change in production
Jwt:Issuer	AuthService	MyApp	Token issuer identifier
Jwt:Audience	AuthService	MyAppUsers	Token audience identifier
Cloudinary:CloudName	ProductService	Your Cloudinary name	Media cloud account name
Cloudinary:ApiKey	ProductService	Your API key	Cloudinary API key
Cloudinary:ApiSecret	ProductService	Your secret	Cloudinary API secret

6. Complete API Reference

6.1 Authentication Endpoints (via Gateway: localhost:5000)

Method	URL	Request Body	Response
POST	/api/auth/register	{"name":"Jane","email":"jane@ex.com","password":"Pass@1","role":"User"}	200: {message} | 409: email exists
POST	/api/auth/login	{"email":"jane@ex.com","password":"Pass@1"}	200: {token, refreshToke user}
POST	/api/auth/refresh-token	{"refreshToken":"base64string"}	200: {token, refreshToke user}
POST	/api/auth/logout	{"refreshToken":"base64string"}	200: {message}
GET	/api/auth/me	Header: Authorization: Bearer {jwt}	200: {userId name, email role}

6.2 Product Endpoints (via Gateway: localhost:5000)

Method	URL	Auth Required	Response
GET	/api/products	None	200: Array of Product objects
GET	/api/products/{id}	None	200: Product | 404: not found
POST	/api/products	Admin JWT	200: 'Product created successfully!'
POST	/api/products/upload	Admin JWT	200: 'Product created successfully!'
PUT	/api/products/{id}	Admin JWT	200: 'Updated successfully!' | 404
PUT	/api/products/upload/{id}	Admin JWT	200: 'Updated successfully!' | 404
DELETE	/api/products/{id}	Admin JWT	200: 'Deleted successfully' | 404

6.3 Order Endpoints (via Gateway: localhost:5000)

Method	URL	Auth Required	Response
POST	/api/orders	User/Admin JWT	200: OrderResponse with orderId and totalAmount
GET	/api/orders	User/Admin JWT	200: Orders array (filtered by role)

Method	URL	Auth Required	Response
GET	/api/orders/{id}	User/Admin JWT	200: Order | 403: Forbidden | 404: Not found
PUT	/api/orders/{id}	Admin JWT	200: 'Order updated successfully' | 404
DELETE	/api/orders/{id}	Admin JWT	200: 'Order deleted successfully!!' | 404

7. Unit Testing

7.1 Test Coverage

All three microservices include dedicated unit test projects using xUnit and Moq frameworks.

Test Project	Coverage Areas
AuthService.Tests	AuthControllerTests: Register (success, duplicate, invalid), Login (valid, invalid password, unknown email), RefreshToken, Logout, GetCurrentUser — JwtAuthServiceTests: RegisterAsync, GenerateToken (claims validation), RefreshToken generation, SaveRefreshTokenAsync
ProductService.Tests	ProductsControllerTests: GetAll, GetById (valid, invalid, not found), Create, CreateWithImage, Update, UpdateWithImage, Delete — ProductServiceImplTests: validation rules — ProductRepositoryTests: CRUD operations with InMemory DB
OrderService.Tests	OrdersControllerTests: Create, GetAll (user vs admin role), GetById (own order, forbidden), Update, Delete — OrderServiceImplTests: CreateOrderAsync (empty items, product not found, totals calculation), GetAll, GetById, UpdateAsync, DeleteAsync

7.2 Running Tests

15. Navigate to any service test directory: cd EcommerceApp/AuthService.Tests
16. Run: dotnet test
17. Or run all tests from solution root: dotnet test EcommerceApp/

8. Future Enhancements

8.1 Backend Enhancements

- RabbitMQ / Azure Service Bus: Async order-confirmation emails and inventory updates between services
- Redis Cache: Cache popular product listings to reduce SQL Server load
- Elasticsearch: Full-text product search with filters, price range, and category facets
- Payment Gateway Integration: Razorpay or Stripe for real payment processing
- API Rate Limiting: Protect endpoints from abuse via Ocelot rate limiting middleware
- Distributed Tracing: Implement Jaeger or Application Insights for microservice observability
- Stock Service: Dedicated inventory management with real-time stock reservations during checkout
- Review & Rating Service: New microservice for product reviews and star ratings
- Coupon Service: Discount code validation and promotion management
- Kubernetes Deployment: Migrate from Docker Compose to K8s for auto-scaling and self-healing

8.2 Security Enhancements

- HTTPS enforcement and SSL termination at Gateway level
- Secrets management with Azure Key Vault or HashiCorp Vault
- OAuth2 / Google social login integration
- IP-based rate limiting and brute-force protection on auth endpoints
- Two-factor authentication (OTP via SMS or email)

8.3 DevOps Enhancements

- CI/CD pipeline with GitHub Actions: automated build, test, and Docker publish
- Health check endpoints (/health) for all services
- Centralized logging with ELK Stack (Elasticsearch, Logstash, Kibana)
- Blue-green deployment strategy for zero-downtime releases

9. Technology Stack Summary

Component	Technology	Version	Purpose
Backend Framework	ASP.NET Core Web API	.NET 8	RESTful microservice APIs
API Gateway	Ocelot	Latest	Request routing and load balancing
Authentication	JWT Bearer + BCrypt	—	Secure token-based auth
ORM	Entity Framework Core	8.x	Database access and migrations
Database	Microsoft SQL Server	2022	Relational data persistence
Image Hosting	Cloudinary	Latest SDK	Product image upload and CDN delivery
Containerization	Docker + Docker Compose	Latest	Service orchestration
Testing	xUnit + Moq + InMemory DB	Latest	Unit testing all services
API Documentation	Swagger / OpenAPI	3.0	Interactive API explorer per service
Validation	FluentValidation + DataAnnotations	—	Request and model validation